BharatText

Indian Language Text Analyzer

Project Report

Name: Roshan P A

Roll No.: 2025BCS0137

Live application: https://bharattext.streamlit.app/

Technology: Python, regex, pandas, Altair, Streamlit

Contents

1. Abstract
2. Introduction and Motivation
3. Objectives
4. System Overview and Architecture
5. Methodology (Tokenization, Sentence Splitting, Graphemes, Script Detection, Chillu Normalization, Stopwords)
6. Statistics and Comparison Metrics
7. User Interfaces and Deployment
8. Design Decisions
9. Testing
10. Known Limitations
11. Future Work
12. Conclusion
13. References

1. Abstract

BharatText is a text analysis tool for Indian languages. It counts words, sentences and paragraphs, computes word and word-pair frequencies, detects the writing script, compares two documents and exports results to CSV. It supports English, Hindi, Tamil and Malayalam directly, and detects eleven scripts in total. It is available as a Streamlit web application (hosted at https://bharattext.streamlit.app/) and as a command-line tool.

Most general-purpose text tools are built around English and break on Indic text because Indian scripts are written with combining marks. BharatText is built around Unicode correctness first, so that a word such as हिन्दी is one word and क्षि is one visible letter.

2. Introduction and Motivation

India has 22 scheduled languages written in more than ten scripts. Indic scripts are *abugidas*: a consonant carries an inherent vowel, which vowel signs (matras) modify, and a virama joins consonants into conjuncts. In Unicode these signs are separate code points of category M (combining marks) attached to a base letter of category L.

This causes three concrete problems for naive tools:

- **Wrong word boundaries.** Python's `\w` does not match combining marks, so `re.findall(r"\w+", "हिन्दी")` returns fragments instead of one word.
- **Wrong character counts.** One visible letter such as क्षि is several code points.
- **Wrong sentence boundaries.** Hindi uses the danda (।, U+0964) as a full stop, and Urdu uses U+06D4 and U+061F.

BharatText addresses these directly and wraps the solution in a usable interface.

3. Objectives

- Tokenize Indic text into correct words without splitting at vowel signs or viramas.
- Split sentences on both Latin and Indic terminators while keeping decimals such as `3.14` intact.
- Count visible letters (grapheme clusters) rather than code points.
- Detect the script of a text, including code-mixed text.
- Provide frequency, statistics and two-document comparison.
- Export results in a form Excel opens correctly.
- Offer both a web interface and a command-line interface.

3.1 Feature summary

Feature	Description
Unicode-correct tokenization	Words are never broken at vowel signs or viramas.
Sentence splitting	Handles `. ! ?`, Devanagari danda U+0964 and U+0965, Urdu U+06D4 and U+061F.
Grapheme counting	क्षि counts as one letter, not four code points.
Script detection	Devanagari, Bengali, Gurmukhi, Gujarati, Odia, Tamil, Telugu, Kannada, Malayalam, Urdu, Latin; works on code-mixed text.
Chillu normalization	Old-style (consonant + virama + ZWJ) and atomic (ൽ) Malayalam chillus count as the same word.
Stopword filtering	English, Hindi, Tamil and Malayalam.
Statistics	Type-token ratio, hapax legomena, average and standard deviation of sentence length, word-length distribution.
Compare two texts	Vocabulary overlap (Jaccard) and frequency similarity (cosine).
CSV export	UTF-8 with BOM so Excel displays Indian scripts correctly.

4. System Overview and Architecture

The project is a small Python package with a thin interface layer on top. All text logic lives in the package, so the web app and the command line share exactly the same code.

```
bharattext/
├── app.py                  Streamlit web interface
├── bharattext/
│   ├── analyzer.py         tokenize, statistics, frequency, compare, CSV export
│   ├── scripts.py          Unicode block → script detection
│   ├── stopwords.py        stopword lists per script
│   └── __main__.py         command-line interface
├── samples/                English, Hindi, Tamil, Malayalam, code-mixed texts
├── tests/test_analyzer.py  pytest suite
└── .streamlit/config.toml  theme
```

Module	Responsibility
`analyzer.py`	Core engine: tokenization, sentence and paragraph splitting, grapheme counting, frequency tables, statistics, document comparison, CSV export.

`scripts.py`	Maps characters to scripts using Unicode block ranges and reports the dominant and mixed scripts.
`stopwords.py`	Holds the stopword lists for each supported language.
`__main__.py`	Command-line entry point: reads a file or stdin, prints the analysis, optionally writes CSV.
`app.py`	Streamlit interface: input, results, charts, comparison, download.

Data flow. Raw text enters from a file, stdin or the web form. It is normalized, then split into paragraphs, sentences and words. Words feed the frequency, pair and statistics routines, while the raw characters feed script detection and grapheme counting. Results are shown on screen or written to CSV.

5. Methodology

5.1 Tokenization

An Indic word is a base letter (Unicode category L) followed by any number of combining marks (category M), joined with further letters. BharatText therefore uses the third-party `regex` module, which supports `\p{L}`, `\p{M}` and grapheme clusters (`\X`), instead of the built-in `re`.

Input	Built-in `re` with `\w+`	BharatText
हिन्दी	fragments (marks are skipped)	one word
தமிழ்	fragments	one word
മലയാളം	fragments	one word

5.2 Sentence and paragraph splitting

Sentences end at `.`, `!`, `?`, the Devanagari danda (U+0964) and double danda (U+0965), and the Urdu full stop (U+06D4) and Arabic question mark (U+061F). A full stop between digits (for example `3.14`) is not treated as a boundary. Paragraphs are separated by blank lines.

5.3 Grapheme counting

Length in code points is misleading for Indic text. For example क्षि is made of four code points (U+0915, U+094D, U+0937, U+093F) but one letter to the reader. BharatText counts grapheme clusters (`\X`) so that word length and character counts match what a person sees.

5.4 Script detection

Each letter is assigned to a script by its Unicode block, and the script with the largest share of letters is reported as dominant. Scripts and their Unicode blocks:

Script	Unicode block	Typical languages
Devanagari	U+0900 – U+097F	Hindi, Marathi, Nepali, Sanskrit
Bengali	U+0980 – U+09FF	Bengali, Assamese
Gurmukhi	U+0A00 – U+0A7F	Punjabi
Gujarati	U+0A80 – U+0AFF	Gujarati
Odia	U+0B00 – U+0B7F	Odia

Tamil	U+0B80 – U+0BFF	Tamil
Telugu	U+0C00 – U+0C7F	Telugu
Kannada	U+0C80 – U+0CFF	Kannada
Malayalam	U+0D00 – U+0D7F	Malayalam
Arabic (Urdu)	U+0600 – U+06FF	Urdu
Latin	U+0041 – U+024F	English and others

> **Script is not language.** Devanagari is used for Hindi, Marathi, Nepali and Sanskrit, so the application reports the script and lists the languages it may be, rather than claiming a single language.

Because every letter is classified, code-mixed text such as Hindi with English words is reported with both scripts and their proportions.

5.5 Malayalam chillu normalization

Malayalam chillu letters can be encoded in two ways: the older sequence consonant + virama + ZWJ (for example N + virama + ZWJ) and the newer atomic code points (for example ൻ). The two look the same but are different strings. BharatText normalizes them to one form before counting, so the same word is not split into two entries in the frequency table.

5.6 Stopword filtering

Common function words can be removed from frequency results for English, Hindi, Tamil and Malayalam. Filtering is optional (`--stopwords` on the command line, a toggle in the web app). The lists are short and hand-picked.

6. Statistics and Comparison Metrics

6.1 Descriptive statistics

Metric	Meaning
Words, sentences, paragraphs	Basic counts of each unit.
Type-token ratio (TTR)	Unique words divided by total words; a simple measure of vocabulary richness.
Hapax legomena	Words that occur exactly once.
Sentence length	Average and standard deviation, in words.

Word-length distribution	Histogram of word lengths in graphemes.
Top words and word pairs	Most frequent words and adjacent word pairs (bigrams).

6.2 Comparing two documents

Jaccard similarity measures vocabulary overlap between the word sets A and B:

$$J(A, B) = |A \cap B| / |A \cup B|$$

Cosine similarity compares the word-frequency vectors u and v of the two texts:

$$cos(u, v) = (u \cdot v) / (\|u\| \|v\|)$$

Jaccard ignores how often a word appears; cosine takes frequency into account. Reporting both gives a fuller picture of how alike two documents are.

6.3 CSV export

Results are exported as UTF-8 with a byte-order mark (BOM). Without the BOM, Excel often assumes a legacy encoding and shows Indian scripts as garbled text.

7. User Interfaces and Deployment

7.1 Web application

The Streamlit app lets the user paste or upload text, view all counts and statistics, see frequency charts, compare two texts and download the CSV. It is deployed on Streamlit Community Cloud and is available at **https://bharattext.streamlit.app/**. To run it locally:

```
pip install -r requirements.txt
streamlit run app.py            # http://localhost:8501
```

7.2 Command line

```
python -m bharattext samples/hindi.txt
python -m bharattext samples/tamil.txt --top 15 --stopwords --csv reports/
echo "Python is easy to learn. Python is powerful." | python -m bharattext -
```

Passing `-` as the file name reads from standard input, so the tool can be used in shell pipelines.

8. Design Decisions

8.1 `regex` instead of `re`

The built-in `re` module cannot match combining marks with `\w` and has no grapheme-cluster support. The `regex` module offers `\p{M}` and `\X`, which are exactly what Indic text processing needs.

8.2 Altair instead of Matplotlib for charts

Matplotlib's default renderer does not perform complex-script shaping, so Malayalam and Tamil conjuncts appear broken in chart labels. Altair charts are drawn by the browser, which shapes Indian scripts correctly.

8.3 One shared engine for both interfaces

All analysis lives in `analyzer.py`. The Streamlit app and the command line only handle input and display, which keeps behaviour identical and makes the engine easy to test.

8.4 Reporting script, not language

Script is determined reliably from Unicode ranges, while language identification between languages sharing a script is much harder. The app reports what it can know with certainty and lists the possible languages.

9. Testing

The project includes a pytest suite in `tests/test_analyzer.py` and sample inputs for English, Hindi, Tamil, Malayalam and code-mixed text in `samples/`. Run the tests with:

```
python -m pytest -q
```

The tests target the behaviours that are easy to get wrong for Indic text: word boundaries with vowel signs and viramas, danda sentence splitting, decimals not ending sentences, grapheme counting, chillu normalization, script detection on mixed text, and the comparison metrics.

10. Known Limitations

- Abbreviations such as "Dr." are treated as sentence ends.
- Words are counted as written. Inflected forms (भाषा / भाषाओं, மொழி / மொழியை) are counted as separate words; there is no stemming or lemmatization.
- Legacy non-Unicode Indian fonts (ISCII and font-specific encodings) are not supported. Files must be UTF-8 or UTF-16.

- Stopword lists are short and hand-picked.
- Language identification is not performed; only the script is detected.

11. Future Work

- Stemming and lemmatization using the Indic NLP Library or Stanza, so inflected forms are grouped.
- Named-entity recognition and part-of-speech tagging.
- Transliteration between scripts.
- Language identification that separates languages sharing a script, such as Hindi and Marathi.
- Sentiment analysis with an Indic language model.
- Abbreviation handling for sentence splitting, and larger, curated stopword lists.

12. Conclusion

BharatText shows that correct Indic text analysis starts with correct Unicode handling. By treating combining marks as part of words, counting grapheme clusters, recognising Indic sentence terminators, normalizing Malayalam chillus and detecting scripts per letter, it gives accurate counts and statistics on text where general tools fail. The shared engine behind the web app and the command line keeps the project small, testable and easy to extend toward stemming, tagging and language identification.

13. References

- The Unicode Standard, chapters on South Asian scripts and the Malayalam block; Unicode Standard Annex #29 (Text Segmentation, grapheme clusters).
- Python `regex` module documentation (Unicode properties and `\X`).
- Streamlit documentation: https://docs.streamlit.io
- Altair documentation: https://altair-viz.github.io
- Indic NLP Library: https://github.com/anoopkunchukuttan/indic_nlp_library
- Live application: https://bharattext.streamlit.app/